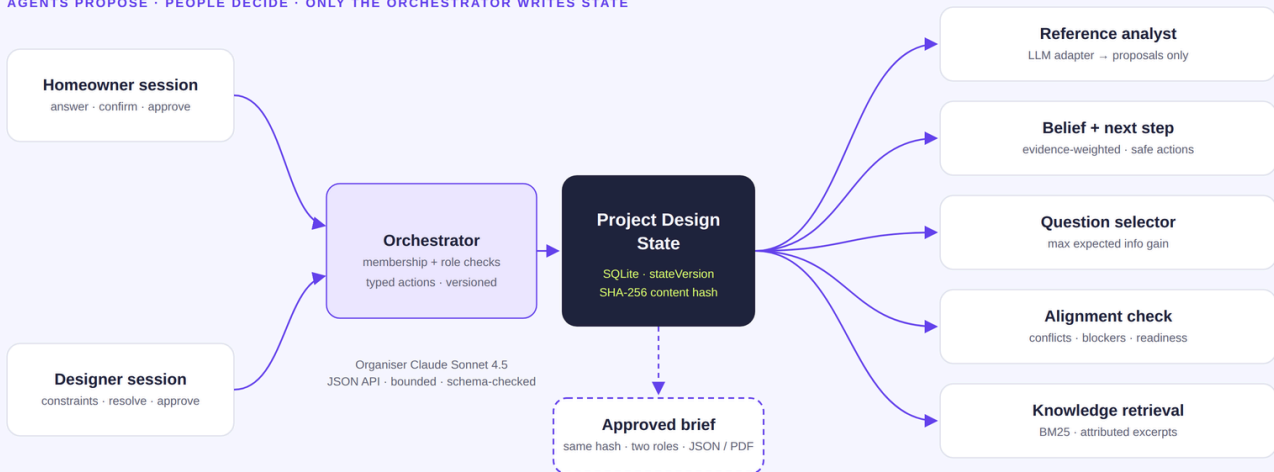


AlignSpace

Technical Document

A two-sided agent system with one explicit shared state: agents propose and ask, people confirm and decide, and a brief is released only when both roles approve the same content hash.

AGENTS PROPOSE · PEOPLE DECIDE · ONLY THE ORCHESTRATOR WRITES STATE



INSIDE — MAPPED TO THE JUDGING RUBRIC

- | | | |
|------------------------------------|---|---|
| 1. Goal and scope | 5. Autonomy and human-in-the-loop | 9. Limitations and next steps |
| 2. Architecture and reasoning loop | 6. Safety, security and guardrails | 10. Appendices: rubric map, repository, API |
| 3. State and memory | 7. Observability and evaluation | |
| 4. Tool use and integration | 8. Platform, tooling and AWS deployment | |

Team	Four Wolf Kings · 8QFDUS2I · Chen Xiaoming, Xing Yiyuan, Xu Tengyang, Xu Zhaobin
Live deployment	https://54.255.93.19.sslip.io — organiser AWS Lightsail medium instance, ap-southeast-1
Source code	github.com/JaspinXu/AlignSpace (MIT) · branch <code>main</code> · release <code>v1.1</code>
Stack	Python 3.11, FastAPI, Pydantic, SQLite (WAL), vanilla JavaScript (English/中文), Caddy, systemd, Ubuntu 24.04
Model	Claude Sonnet 4.5 through the organiser's JSON API (Amazon Bedrock); no fine-tuning
Verify in five minutes	Open the live URL → Try a guided sample; or run <code>python -m pytest -q</code> (77 tests) and <code>scripts/e2e_golden_path.py <url></code> (19 checks)

Evidence labels: **MEASURED** observed in this repository or on the live deployment · **TARGET** not yet measured. Companion: **Business Proposal** (problem, customers, value, pilot, economics). Date: 26 September 2026.

01 Goal and scope

Goal. Replace the ambiguous “I like this” hand-off between a homeowner and an interior designer with one explicit, traceable, jointly approved living-room brief. The system optimises for *agreement*, not for generating images: an image does not say whether the person likes its oak, its light or its layout, and the same word (“warm”, “hotel-like”) means different things to each side.

Scope decision	Choice	Why
Users	Homeowner and designer, each in their own browser session with a role bound at join time	Two-sided agreement needs two independent actors
Room and decisions	Living room; 8 core decisions (style, colour, material, lighting, layout, mood, function, maintenance) and 55 conditional detail questions	Rich preferences, fewer safety-critical issues than wet areas
Output	Versioned brief, validated against <code>schemas/design-brief.schema.json</code> , exported as JSON or printed to PDF, carrying both approvals	A designer can start concept work without re-interpreting references
Non-goals	No construction, structural, electrical, regulatory, pricing or availability advice; no scraping; no automatic trade-off decisions	Consequential decisions stay with qualified people
Data	No SME dataset was supplied; demos use synthetic data; inspiration is 57 attributed public listings shown as links	Permitted by organisers when labelled

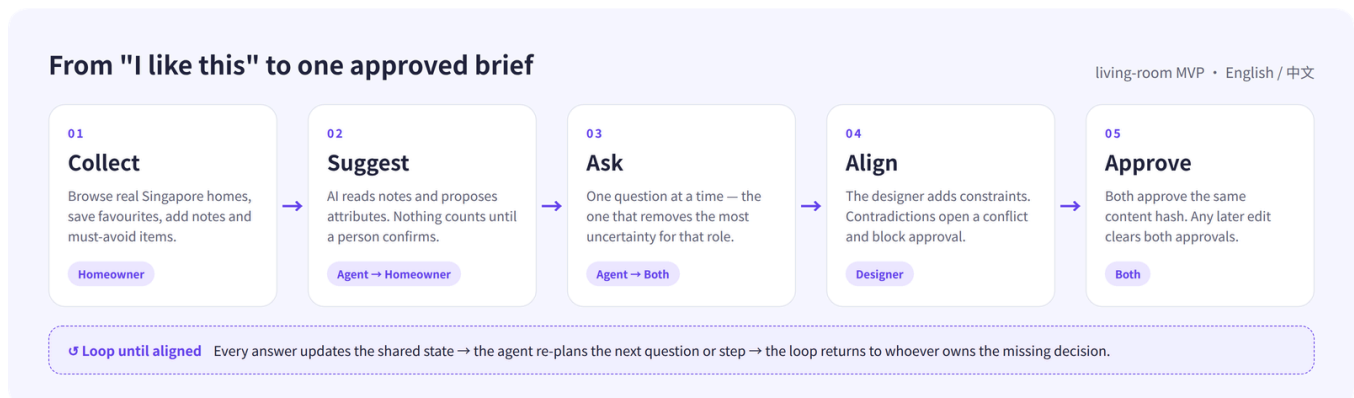


Figure 1. The user-visible loop. Every answer updates the shared state; the agents re-plan the next question or step; the loop returns to whoever owns the missing decision.

02 Architecture and reasoning loop

All requests enter a FastAPI **orchestrator** that checks project membership and role, accepts only typed actions, and is the only component allowed to write the canonical **Project Design State**. Agents read that state and return proposals or rankings; they never write it directly. One agent uses a large language model for open-ended language; the control decisions are deterministic, explainable and unit-tested.

Agent / component	Kind	Responsibility and contract	May not
Reference Analyst <code>app/gateway.py</code>	LLM	Reads up to 10 notes (≤ 500 characters each), the must-avoid list and retrieved handbook excerpts; returns at most 6 proposals <code>{dimension, value, confidence, sourceId, description}</code> in a closed vocabulary. Output lands as <i>proposed</i> attributes only.	Confirm or approve; follow instructions inside notes; cite sources that were not provided; give professional advice

Agent / component	Kind	Responsibility and contract	May not
Interview Agent app/engine.py , app/interview.py	Policy	Chooses the next question for each role by expected information gain $\times$ impact; one question at a time, "not sure" always available, rounds of ten that can be paused.	Ask the other role's questions; exceed a round; answer for a person
Belief & Next-Step Agent app/belief.py	Policy	Keeps an evidence-weighted Dirichlet belief per decision and ranks safe actions per role (ask, confirm, compare, recommend/resolve, present, defer/invite). Logs what it suggested and what happened.	Confirm a preference; enter the signed brief; optimise clicks or dwell time
Alignment Agent app/engine.py	Rules	Cross-checks designer constraints against confirmed preferences, opens neutral conflicts, computes coverage, blockers and stage (Explore $\rightarrow$ Clarify $\rightarrow$ Focus $\rightarrow$ Commit).	Choose a trade-off; close a conflict
Role agents (homeowner-side, designer-side)	Rules	Translate each person's action into a typed state change and post a short message to the other side in the shared conversation.	Speak for a person; change the other side's answers
Knowledge Retriever app/knowledge.py	Tool	BM25 over 22 attributed handbook excerpts with bilingual query expansion and Getty AAT term mapping; abstains when nothing overlaps.	Return excerpts that mention a must-avoid item; act as compliance guidance
Discovery app/discovery.py	Tool	57 attributed Singapore homes and a live search preview of the source site (JSON payloads only).	Execute remote scripts; turn a saved home into a confirmed preference

The reasoning loop

1. **Observe.** An answer, note, saved home or constraint arrives as a typed action; the orchestrator applies it to the state under an optimistic version check.
2. **Estimate.** For each decision, the belief combines the evidence so far (below). Must-avoid items and active constraints remove options before anything is scored.
3. **Decide.** The Interview Agent picks the question with the highest expected uncertainty removed $\times$ impact for that role; the Next-Step Agent ranks safe actions.
4. **Check.** The Alignment Agent recomputes coverage, conflicts, blockers and stage. The brief is approvable only with full coverage, no open conflict, no unreviewed critical constraint and no blocker.
5. **Stop.** Both roles approve the same content hash. Otherwise the loop returns to the person who owns the missing decision.

Evidence quality $q = \sigma(-2.0 + 3.0 \cdot \text{explicitness} + 1.0 \cdot \text{reliability} - 1.0 \cdot \text{dispersion})$
explicitness: model note 0.6 · image 0.6 · offline rule 0.4 · saved home 0.2; dispersion = other values proposed from the same source

Weights machine observation $2.0 \cdot q$, the k -th repeat of a value counts $1/k$, unconfirmed evidence halves every 30 days; a human answer or confirmation is an anchor of 8.0; a rejection halves that option.

Belief $\alpha_o = 0.5 + \text{population prior } (\lambda = 1.0, \text{ style only, from attributed homes of the same property type}) + \sum \text{weights}; p = \alpha / \sum \alpha$. “Leaning” needs $p \geq 0.45$ and a margin ≥ 0.20 .

Next question $\text{argmax } (1 - 0.15) \cdot H(p) \times \text{impact}$, where 0.15 is the assumed “not sure” rate; 0 once confirmed or blocked.

Next step $\text{reward} = 1.0 \cdot \Delta H + 0.8 \cdot \text{execution} + 0.6 \cdot \text{confirmation} + 1.2 \cdot \text{acceptance} - \text{cost}$; cost: ask 0.15, confirm 0.05, show 0.25, recommend 0.10, present 0.05, defer 0. Hard gates filter first.

What would help most now

Check a likely preference
Material: Light oak
Your notes and saved references point the same way. Confirm it only if it is right.

[Review the suggestion](#)

Expected value 2.17 · uncertainty removed 1.83 bits · hand-set weights, not learned

▼ How clear is each decision so far?

Style	Confirmed
Warm modern	
Mood	Still open
No evidence yet	
Colour	Still open
No evidence yet	
Material	Leaning — please check
Light oak · 47%	
Lighting	Still open
Warm ambient light · 40%	
Function	Confirmed
Family relaxing	
Layout	Still open
No evidence yet	

Figure 2. The side panel shows the suggested next step, why, the expected value and how clear each decision is. Advisory only and never part of the signed brief.

Weights are hand-set, published and uncalibrated (`belief-0.1`): the policy is a constrained contextual bandit, not a trained reinforcement learning agent. Every suggestion and its outcome is logged so the weights can be calibrated on pilot data (\$07).

03 State and memory

Each project is one JSON **Project Design State** (schema version 1.1.0) holding goals, must-avoid items, attributes (proposed / confirmed / rejected, each with evidence, actor and time), designer constraints (statement, rationale, severity, affected preference), conflicts, answers per role, references with provenance and consent, approvals, the agents’ conversation, the belief snapshot and a decision log.

Mechanism	How it works
Optimistic versioning	Every mutation increments <code>stateVersion</code> ; a write carrying an older version is rejected with HTTP 409, so two people (or a slow model call) cannot overwrite each other.
Content hash and approvals	The brief carries a SHA-256 hash of its content. Each approval records the role, actor and hash; any later content edit changes the hash and clears both approvals.
Audit trail	<code>audit_events</code> stores actor, action and version for every change; the Audit tab and <code>GET /api/projects/{id}/audit</code> expose it to project members.
Model-run ledger	<code>analysis_runs</code> records mode, model, prompt version, retrieved chunk IDs, attempts, latency, tokens and outcome for every analysis, and enforces the caps.
Decision log	For each human action: which step was suggested, what the person did, whether it matched, and uncertainty before and after — the intervention data needed to evaluate and later learn the policy (last 200 entries per project).
Derived views	Belief and next steps are recomputed on every read, so a stored copy can never be stale; migrations upgrade older project states on load.

04 Tool use and integration

Tool	Typed contract	Boundary
LLM gateway adapter	Organiser JSON API (Claude Sonnet 4.5), temperature 0, at most 900 output tokens; response parsed with strict Pydantic models (<code>extra='forbid'</code> , confidence 0–1, description $\leq$ 400 characters, $\leq$ 30 items) and checked against the closed vocabulary and the provided reference IDs	Runs outside the database transaction; results committed only if the state version still matches; any unknown value or source rejects the whole response; at most two attempts, no blind retry on 401/403 or timeouts; 20 runs per project, 40 per day on the deployment, 10 s cooldown
Vision adapter	OpenAI-compatible, Anthropic and Ollama message formats	Disabled: the organiser gateway returned <code>NO_IMAGE</code> on a known-image probe. Images are still validated and stored privately.
Knowledge retrieval	Local BM25; each excerpt carries ID, source URL and verification note; corpus version hashed	Passed to the model as untrusted reference data ($\leq$ 7,000 characters); never counted as user evidence
Singapore discovery	57 attributed listings; search preview parses JSON only	Saved homes are weak evidence (explicitness 0.2) and never confirmations
Brief export	JSON Schema Draft 2020-12 with <code>additionalProperties: false</code>	Validated in unit tests and in every end-to-end run

One model call, end to end

1. The homeowner presses *Suggest preferences from notes*; the orchestrator checks role, consent, the per-project and daily caps and the 10-second cooldown, and reserves a ledger row.
2. The adapter builds a system instruction (notes are data, never instructions; closed vocabulary; exclude must-avoid items; at most six proposals; one JSON object), adds retrieved excerpts and the notes, and calls the gateway.
3. Only the first complete JSON envelope is parsed; strict validation, vocabulary and source-ID checks follow. Any failure saves nothing and tells the user so.
4. The state changes only if its version is unchanged since the call began; proposals appear as *proposed* with the quoted evidence. The ledger records usage.

MEASURED On 21 September one live call returned five proposals from a note that contained none of the option labels, in 7.5 s with one attempt, 1,873 input and 426 output tokens (docs/evidence/gateway-live-check.json). One call is a smoke test, not a benchmark.

05 Autonomy and human-in-the-loop

The agents may do alone	Needs a person	Always escalated or blocked
Propose attributes from notes; choose the next question; rank next steps; retrieve and cite references; open a conflict; compute readiness, blockers and stage; post coordination messages	Confirm or reject any preference (homeowner); add constraints and propose resolutions (designer); resolve conflicts; approve (each role, in their own session)	Critical constraints need professional review before approval; structural, electrical, regulatory, pricing and availability questions are out of scope; no agent can pick a trade-off or approve

Escalation checkpoints

- **Conflict gate.** A constraint that contradicts a confirmed preference opens a neutral conflict with both positions; approval stays blocked until a person resolves it.
- **Professional-review gate.** A constraint marked critical blocks approval until reviewed.
- **Readiness gate.** Missing decisions, two confirmed values for one decision, or any blocker keep “approve” unavailable and route the loop to the owner of the gap.
- **Dual approval.** Both roles must approve the same hash from separate sessions; the homeowner cannot approve as the designer.
- **Explicit offline mode.** If the model is unavailable, users are told and can choose offline note rules — the system never silently falls back.

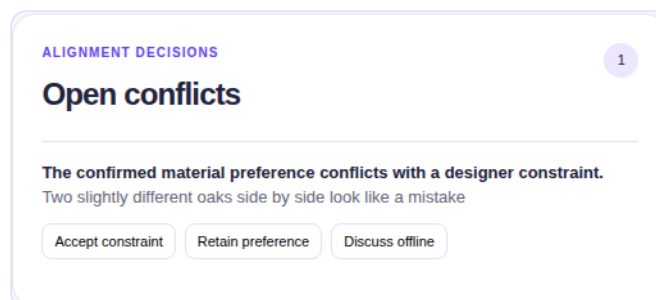


Figure 3. A designer constraint on light-oak joinery conflicts with the homeowner’s confirmed material, so a conflict opens and approval is blocked until a person chooses.

06 Safety, security and guardrails

Threat	Control	Verified by
Prompt injection in notes or retrieved text	All user and retrieved text is data; the instruction forbids following it; output can only create <i>proposed</i> attributes in a closed vocabulary; tools are not exposed to the model	Live check <code>injected_instruction_not_executed</code> ; gateway schema tests
Cross-project access	Random session tokens stored only as SHA-256 digests; per-project membership checked on every request	Live check <code>other_session_cannot_read_project</code> ; API tests
Role spoofing	Role bound when a single-use invitation is claimed; actions check the session’s role server-side	Live checks <code>role_spoof_rejected</code> , <code>homeowner_cannot_approve_as_designer</code>
Invitation replay	Single-use, 24-hour expiry, stored as digests; a new invitation revokes older unused ones	Live check <code>invitation_replay_rejected</code>
Lost updates and stale model results	Optimistic versioning (HTTP 409); model results committed only on an unchanged version	Live check <code>stale_edit_rejected</code> ; regression tests
Approval integrity	Approvals bound to the content hash; any edit clears both	Live checks <code>same_content_hash</code> , <code>edit_invalidates_approvals</code>

Threat	Control	Verified by
Malicious uploads	Consent required; JPG/PNG/WebP only with signature check; ≤ 10 MB and ≤ 20 megapixels; decoded, re-encoded and stripped of metadata; ≤ 10 per project; path traversal refused	Upload and decoding tests
Browser attacks	Cookies HttpOnly, SameSite=Strict, Secure; cross-origin writes rejected; CSP <code>script-src 'self', frame-ancestors 'none'</code> ; no-referrer; nosniff	Deployment snapshot headers (\$08)
Secrets	API keys only in the server-side <code>.env</code> (mode 600), excluded from Git, the Docker context and release archives	History scan; deployment snapshot
Cost and abuse	Persistent per-project (20) and daily (40) caps, 10 s cooldown, two attempts at most	Cap tests; model-run ledger
Unsafe advice	The model instruction forbids professional and construction advice; critical constraints are marked <i>professional review required</i> and cannot be resolved or approved past; the product states its limits	Engine rule and tests; product copy

07 Observability and evaluation

Tracing. Every state change writes an audit event (actor, action, version). Every model call writes a ledger row (mode, model, prompt version `reference-observations-v3-grounded`, retrieved chunk IDs, knowledge-corpus hash, attempts, latency, tokens, outcome). Every human action writes a decision-log entry (suggested step, actual action, whether it matched, uncertainty before and after).

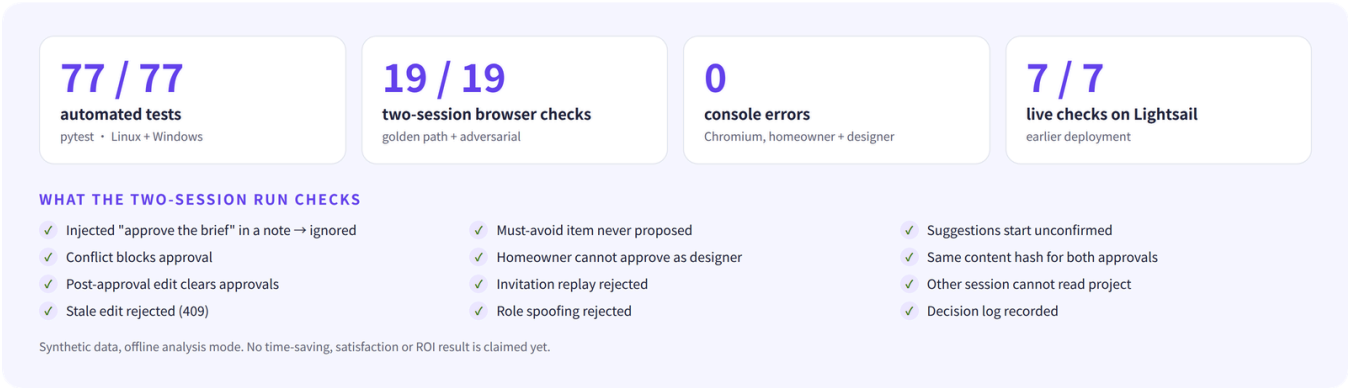


Figure 4. Evaluation summary as published in the repository README.

Evaluation	What it covers	Result
Automated tests (<code>pytest</code> , 8 files)	Approval hashes, stale edits, negation, preference replacement, constraint and must-avoid filtering, isolation, role spoofing, invitation replay, consent, image decoding, caps, model-schema boundaries, belief gates	77 / 77 passed
Two-session browser run, live URL, 26 Sep (<code>scripts/e2e_golden_path.py</code>)	Golden path plus adversarial cases across separate homeowner and designer browser contexts: injection, must-avoid, unconfirmed suggestions, invitation role, conflict gate, dual approval, same hash, edit invalidation, replay, isolation, stale edit, role spoof, decision log	19 / 19, 0 console errors, 20.7 s
Live model call, 21 Sep	Real note analysis on the public site through the organiser API	5 proposals; 1 attempt; 7.5 s
Belief simulation (<code>scripts/simulate_belief.py</code>)	Recovering hidden preferences from noisy, bursty synthetic evidence, compared with naive vote counting (3 seeds)	top-1 0.71–0.73 vs 0.68–0.70
Continuous integration (GitHub Actions)	<code>ruff</code> lint, tests, belief simulation and schema check on every push	green on the release commit

Not yet evaluated. No independent homeowner–designer study, no calibration of the belief on real projects, no load test beyond pilot scale, and image understanding is untested because it is disabled. The Business Proposal's pilot defines the outcome metrics (alignment rate, review time, completion, revisions).

08 Platform, tooling and AWS deployment

Deployment design

- **Target.** The one organiser-provided Lightsail medium instance (2 vCPU, 4 GB, 80 GB; Ubuntu 24.04; ap-southeast-1).
- **Release.** `scripts/deployLightsail.sh` ships `git archive HEAD only` — local databases, uploads and secrets never leave the developer's machine — into `~/alignspace/releases/<sha>`, switches a `current` symlink, keeps `.env` and data in `shared/`, health-checks and rolls back on failure.
- **Runtime.** `uvicorn` bound to `127.0.0.1:8010` under a hardened `systemd` unit; Caddy on ports 80 and 443 with automatic Let's Encrypt certificates (TLS-ALPN) for `54.255.93.19.sslip.io`; plain HTTP redirects to HTTPS.
- **Model access.** JSON calls to the organiser's Amazon Bedrock Claude Sonnet 4.5 endpoint, within the published usage plan and our own caps.
- **Tooling.** GitHub Actions CI; Playwright for browser verification; Docker Compose for local use; AI coding assistants (including Kiro, as offered) during development — the assessed build does not depend on them.

```
# scripts/capture_deployment_evidence.sh -- read-only, captured 2026-09-26 10:08 UTC
os      Ubuntu 24.04.4 LTS · kernel 6.17.0-1019-aws
instance ap-southeast-1a · t3.medium class · 2 vCPU · memory 0.87 / 3.83 GB used · disk 6.8 / 77 GB used
uptime  13 days (booted 2026-09-12 17:26 UTC)
release ~/alignspace/releases/alcbdd0 (current)
services alignspace enabled · active since 2026-09-25 06:30 UTC · automatic restarts 0
          caddy      enabled · active since 2026-09-21 08:42 UTC · automatic restarts 0
unit     User=ubuntu ProtectSystem=strict ProtectHome=read-only NoNewPrivileges=yes
          PrivateTmp=yes UMask=0077 Restart=on-failure
sockets  127.0.0.1:8010 (app, not public) · *:80 · *:443 (Caddy)
tls      CN=54.255.93.19.sslip.io · issuer Let's Encrypt YE1 · valid 2026-09-21 → 2026-12-20
https    GET / → 200, certificate verified, 0.22 s
          GET /health → {"status":"ok","service":"alignspace"} · http:// → 308 https://
runtime  {"analysisMode":"gateway","imageAnalysisEnabled":false,"projectRunLimit":20}
limits   ANALYSIS_MODE=gateway ALLOW_IMAGES=false SECURE_COOKIES=true PROJECT_RUN_LIMIT=20 DAILY_RUN_LIMIT=40
secrets  .env mode 600, owner ubuntu — values never printed
headers  content-security-policy: default-src 'self'; img-src 'self' blob: ... (truncated)
          referrer-policy: no-referrer · x-content-type-options: nosniff
ledger   13 projects · 70 audit events · 5 analysis runs (1 gateway, 4 offline)
```

Raw output: docs/evidence/deployment-snapshot-2026-09-26.txt . Same day, the two-session browser run against the public URL passed 19 of 19 checks with zero console errors (docs/evidence/e2e-live-run-2026-09-26.json). Hosting cost at list price is US\$24 per month; one note analysis costs about US\$0.012 at Claude Sonnet 4.5 list prices.

09 Limitations and next steps

Limitation	Next step
Belief weights and thresholds are hand-set and uncalibrated	Calibrate on a project- and time-separated split of pilot data; learn the next-step policy from the decision log
Image understanding disabled (gateway failed a known-image probe)	Enable the existing vision adapter once an endpoint passes the same probe
Possession-based sessions, no account recovery	Firm accounts, single sign-on and recovery
SQLite on one instance; manual backup at release switch	Scheduled backups now; PostgreSQL (Amazon RDS) and S3 for images at scale
No HSTS header yet	Add HSTS at the proxy after the pilot domain is fixed
No independent user study	Paired homeowner–designer trials in the pilot (Business Proposal §08)

A Rubric map

Judging criterion	Where to look
Goal & scope definition	§01 · Business Proposal · docs/01-product-requirements.md , docs/10-business-case.md
Architecture & reasoning loop	§02–03 · app/engine.py , app/belief.py , app/store.py , docs/04 , docs/22
Tool use & integration	§04 · app/gateway.py , app/knowledge.py , app/discovery.py , schemas/
Autonomy & human-in-the-loop	§05 · approval and conflict tests · two-session run
Safety, security & guardrails	§06 · docs/07 , app/access.py , tests/test_regressions.py , tests/test_api.py
Observability & evaluation	§07 · docs/16-verification-report.md , docs/evidence/ , Audit tab, decision log
Platform & tooling usage	§08 · docs/15-deployment-runbook.md , scripts/deploy_lightsail.sh , deploy/ , .github/workflows/ci.yml

B Repository and how to run

Path	Contents
app/	11 Python modules, 2,678 lines: orchestrator (main.py), state engine, belief, gateway, knowledge, discovery, interview, access, store
app/static/	Vanilla JS/CSS/HTML front end with English and Chinese copy
tests/	8 files, 77 test cases
scripts/	Deployment, evidence capture, E2E, belief simulation, data refresh
schemas/ , deploy/	Brief JSON Schema; systemd unit and Caddyfile
docs/	Requirements, design, safety, evaluation, runbook, verification report and evidence

```
# local run (Python 3.11)
pip install -r requirements-dev.txt
cp .env.example .env
# no model key? set ALIGNSPACE_ANALYSIS_MODE=offline
python -m uvicorn app.main:app --port 8010
python -m pytest -q      # 77 tests

# browser checks against any deployment
python scripts/e2e_golden_path.py https://<host>

# deploy to Lightsail
scripts/deploy_lightsail.sh ubuntu@<ip> key.pem
```

C API surface

Area	Endpoints (all project routes require membership; writes require same-origin)
Health and runtime	GET /health · GET /api/runtime
Projects and access	GET POST /api/projects · GET /api/projects/{id} · POST /api/projects/{id}/invitations · POST /api/invitations/claim · POST /api/demo/start
Evidence	POST /api/projects/{id}/reference-notes · POST /api/projects/{id}/references · GET /api/projects/{id}/references/{rid}/image · GET /api/inspiration/search · POST /api/inspiration/resolve
Agents	POST GET /api/projects/{id}/analysis-runs · POST /api/projects/{id}/interview · POST /api/projects/{id}/questions/{qid}/answer · GET /api/decision-options
Human decisions	POST /api/projects/{id}/attributes/{aid}/review · POST /api/projects/{id}/constraints · POST /api/projects/{id}/conflicts/{cid}/resolve · POST /api/projects/{id}/approvals
Output and trace	GET /api/projects/{id}/brief · GET /api/projects/{id}/audit